

CSA0503-DATABASE MANAGEMENT SYSTEMS

CO2 AT1

Analytical Problem Solving

1. Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

```
mysql> CREATE TABLE Department (  
->     DeptID INT PRIMARY KEY,  
->     DeptName VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.11 sec)  
  
mysql>  
mysql> CREATE TABLE Employee (  
->     EmpID INT PRIMARY KEY,  
->     EmpName VARCHAR(30),  
->     Salary DECIMAL(10,2),  
->     DeptID INT,  
->     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql> INSERT INTO Department VALUES  
-> (1,'HR'),  
-> (2,'IT'),  
-> (3,'Finance');  
Query OK, 3 rows affected (0.02 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql>  
mysql> INSERT INTO Employee VALUES  
-> (101,'Rahul',50000,1),  
-> (102,'Anita',60000,2),  
-> (103,'Kiran',45000,1),  
-> (104,'Priya',70000,3);  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT *
-> FROM Employee
-> WHERE Salary > 50000;
+-----+-----+-----+-----+
| EmpID | EmpName | Salary | DeptID |
+-----+-----+-----+-----+
| 102 | Anita | 60000.00 | 2 |
| 104 | Priya | 70000.00 | 3 |
+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

```
mysql> SELECT EmpName
-> FROM Employee;
+-----+
| EmpName |
+-----+
| Rahul |
| Anita |
| Kiran |
| Priya |
+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT EmpName
-> FROM Employee
-> WHERE DeptID = 1
->
-> UNION
->
-> SELECT EmpName
-> FROM Employee
-> WHERE DeptID = 2;
+-----+
| EmpName |
+-----+
| Rahul |
| Kiran |
| Anita |
+-----+
3 rows in set (0.01 sec)
```

```
mysql> SELECT EmpName
-> FROM Employee
-> WHERE DeptID = 1
-> AND EmpID NOT IN
-> (
->     SELECT EmpID
->     FROM Employee
->     WHERE DeptID = 2
-> );
+-----+
| EmpName |
+-----+
| Rahul |
| Kiran |
+-----+
2 rows in set (0.01 sec)
```

```
mysql> SELECT *
-> FROM Employee
-> CROSS JOIN Department;
+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | Salary | DeptID | DeptID | DeptName |
+-----+-----+-----+-----+-----+-----+
| 101 | Rahul | 50000.00 | 1 | 3 | Finance |
| 101 | Rahul | 50000.00 | 1 | 2 | IT |
| 101 | Rahul | 50000.00 | 1 | 1 | HR |
| 102 | Anita | 60000.00 | 2 | 3 | Finance |
| 102 | Anita | 60000.00 | 2 | 2 | IT |
| 102 | Anita | 60000.00 | 2 | 1 | HR |
| 103 | Kiran | 45000.00 | 1 | 3 | Finance |
| 103 | Kiran | 45000.00 | 1 | 2 | IT |
| 103 | Kiran | 45000.00 | 1 | 1 | HR |
| 104 | Priya | 70000.00 | 3 | 3 | Finance |
| 104 | Priya | 70000.00 | 3 | 2 | IT |
| 104 | Priya | 70000.00 | 3 | 1 | HR |
+-----+-----+-----+-----+-----+-----+
12 rows in set (0.00 sec)
```

```
mysql> SELECT EmpID, EmpName, Salary, DeptName
-> FROM Employee
-> INNER JOIN Department
-> ON Employee.DeptID = Department.DeptID;
```

EmpID	EmpName	Salary	DeptName
101	Rahul	50000.00	HR
103	Kiran	45000.00	HR
102	Anita	60000.00	IT
104	Priya	70000.00	Finance

4 rows in set (0.00 sec)

2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, deptid) and DEPARTMENT(deptid, deptname) tables with all appropriate constraints.

```
mysql> CREATE TABLE DEPARTMENT (
->     DeptID INT PRIMARY KEY,
->     DeptName VARCHAR(50) NOT NULL UNIQUE
-> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> CREATE TABLE STUDENT (
->     SID INT PRIMARY KEY,
->     Name VARCHAR(50) NOT NULL,
->     DOB DATE NOT NULL,
->     DeptID INT NOT NULL,
->     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
-> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
SID	int	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
DOB	date	NO		NULL	
DeptID	int	NO	MUL	NULL	

4 rows in set (0.01 sec)

```
mysql> DESC DEPARTMENT;
```

Field	Type	Null	Key	Default	Extra
DeptID	int	NO	PRI	NULL	
DeptName	varchar(30)	YES		NULL	

2 rows in set (0.01 sec)

3. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

```
mysql> CREATE TABLE Sales (  
->     SaleID INT PRIMARY KEY,  
->     ProductName VARCHAR(50),  
->     Category VARCHAR(30),  
->     Quantity INT,  
->     Amount DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> INSERT INTO Sales VALUES  
-> (1, 'Laptop', 'Electronics', 2, 100000),  
-> (2, 'Mouse', 'Electronics', 5, 5000),  
-> (3, 'Chair', 'Furniture', 3, 15000),  
-> (4, 'Table', 'Furniture', 2, 20000),  
-> (5, 'Keyboard', 'Electronics', 4, 8000),  
-> (6, 'Sofa', 'Furniture', 1, 30000);  
Query OK, 6 rows affected (0.02 sec)  
Records: 6  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT Category, COUNT(*) AS TotalSales  
-> FROM Sales  
-> GROUP BY Category;  
+-----+-----+  
| Category | TotalSales |  
+-----+-----+  
| Electronics | 3 |  
| Furniture | 3 |  
+-----+-----+  
2 rows in set (0.01 sec)  
  
mysql> SELECT Category, SUM(Amount) AS TotalAmount  
-> FROM Sales  
-> GROUP BY Category;  
+-----+-----+  
| Category | TotalAmount |  
+-----+-----+  
| Electronics | 113000.00 |  
| Furniture | 65000.00 |  
+-----+-----+  
2 rows in set (0.00 sec)
```

```
mysql> SELECT Category, AVG(Amount) AS AverageAmount
-> FROM Sales
-> GROUP BY Category;
```

Category	AverageAmount
Electronics	37666.666667
Furniture	21666.666667

2 rows in set (0.01 sec)

```
mysql> SELECT Category, MAX(Amount) AS HighestSale
-> FROM Sales
-> GROUP BY Category;
```

Category	HighestSale
Electronics	100000.00
Furniture	30000.00

2 rows in set (0.00 sec)

```
mysql> SELECT Category, MIN(Amount) AS LowestSale
-> FROM Sales
-> GROUP BY Category;
```

Category	LowestSale
Electronics	5000.00
Furniture	15000.00

2 rows in set (0.00 sec)

```
mysql> SELECT Category, SUM(Amount) AS TotalAmount
-> FROM Sales
-> GROUP BY Category
-> HAVING SUM(Amount) > 50000;
```

Category	TotalAmount
Electronics	113000.00
Furniture	65000.00

2 rows in set (0.00 sec)

4. Write a correlated sub-query to find employees earning more than the average salary of their own department.

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | Salary   | DeptID |
+-----+-----+-----+-----+
| 101   | Rahul   | 50000.00 | 1      |
| 102   | Anita   | 60000.00 | 2      |
| 103   | Kiran   | 45000.00 | 1      |
| 104   | Priya   | 70000.00 | 3      |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT EmpID, EmpName, Salary, DeptID
-> FROM Employee e
-> WHERE Salary >
-> (
->     SELECT AVG(Salary)
->     FROM Employee
->     WHERE DeptID = e.DeptID
-> );
+-----+-----+-----+-----+
| EmpID | EmpName | Salary   | DeptID |
+-----+-----+-----+-----+
| 101   | Rahul   | 50000.00 | 1      |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

```
mysql> CREATE TABLE Project (  
->     ProjectID INT PRIMARY KEY,  
->     ProjectName VARCHAR(30),  
->     EmpID INT,  
->     FOREIGN KEY (EmpID) REFERENCES Employee(EmpID)  
-> );  
ERROR 1050 (42S01): Table 'project' already exists  
mysql> desc project;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| ProjectID  | int           | NO   | PRI | NULL    |       |  
| ProjectName | varchar(30)   | YES  |     | NULL    |       |  
| EmpID      | int           | YES  | MUL | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.01 sec)  
  
mysql> INSERT INTO Project VALUES  
-> (1,'Payroll',101),  
-> (2,'Website',102),  
-> (3,'AI System',103),  
-> (4,'Mobile App',104);  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT e.EmpID, e.EmpName, p.ProjectName  
-> FROM Employee e  
-> INNER JOIN Project p  
-> ON e.EmpID = p.EmpID;
```

```
+-----+-----+-----+  
| EmpID | EmpName | ProjectName |  
+-----+-----+-----+  
| 101   | Rahul   | Payroll     |  
| 102   | Anita   | Website     |  
| 103   | Kiran   | AI System   |  
| 104   | Priya   | Mobile App  |  
+-----+-----+-----+  
4 rows in set (0.01 sec)
```

```
mysql> SELECT e.EmpID, e.EmpName, p.ProjectName  
-> FROM Employee e  
-> LEFT JOIN Project p  
-> ON e.EmpID = p.EmpID;
```

```
+-----+-----+-----+  
| EmpID | EmpName | ProjectName |  
+-----+-----+-----+  
| 101   | Rahul   | Payroll     |  
| 102   | Anita   | Website     |  
| 103   | Kiran   | AI System   |  
| 104   | Priya   | Mobile App  |  
+-----+-----+-----+  
4 rows in set (0.00 sec)
```

```
mysql> SELECT e.EmpID, e.EmpName, p.ProjectName
-> FROM Employee e
-> RIGHT JOIN Project p
-> ON e.EmpID = p.EmpID;
```

EmpID	EmpName	ProjectName
101	Rahul	Payroll
102	Anita	Website
103	Kiran	AI System
104	Priya	Mobile App

4 rows in set (0.00 sec)

```
mysql> SELECT e.EmpID, e.EmpName, p.ProjectName
-> FROM Employee e
-> LEFT JOIN Project p
-> ON e.EmpID = p.EmpID
->
-> UNION
->
-> SELECT e.EmpID, e.EmpName, p.ProjectName
-> FROM Employee e
-> RIGHT JOIN Project p
-> ON e.EmpID = p.EmpID;
```

EmpID	EmpName	ProjectName
101	Rahul	Payroll
102	Anita	Website
103	Kiran	AI System
104	Priya	Mobile App

4 rows in set (0.00 sec)

```
mysql> SELECT e.EmpID, e.EmpName, p.ProjectName
-> FROM Employee e
-> CROSS JOIN Project p;
```

EmpID	EmpName	ProjectName
104	Priya	Payroll
103	Kiran	Payroll
102	Anita	Payroll
101	Rahul	Payroll
104	Priya	Website
103	Kiran	Website
102	Anita	Website
101	Rahul	Website
104	Priya	AI System
103	Kiran	AI System
102	Anita	AI System
101	Rahul	AI System
104	Priya	Mobile App
103	Kiran	Mobile App
102	Anita	Mobile App
101	Rahul	Mobile App

16 rows in set (0.00 sec)

6. Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

```
mysql> CREATE VIEW DeptSalaryView AS
-> SELECT
->     d.DeptName,
->     COUNT(e.EmpID) AS TotalEmployees,
->     AVG(e.Salary) AS AverageSalary
-> FROM Department d
-> INNER JOIN Employee e
-> ON d.DeptID = e.DeptID
-> GROUP BY d.DeptName;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> SELECT * FROM DeptSalaryView;
+-----+-----+-----+
| DeptName | TotalEmployees | AverageSalary |
+-----+-----+-----+
| HR       | 2              | 47500.000000  |
| IT       | 1              | 60000.000000  |
| Finance  | 1              | 70000.000000  |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

```
mysql> SELECT *
-> FROM DeptSalaryView
-> WHERE AverageSalary > 50000;
+-----+-----+-----+
| DeptName | TotalEmployees | AverageSalary |
+-----+-----+-----+
| IT       | 1              | 60000.000000  |
| Finance  | 1              | 70000.000000  |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

7. Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

```
mysql> SELECT s.Name
-> FROM STUDENTS s
-> JOIN ENROLLMENT e
-> ON s.SID = e.SID
-> GROUP BY s.SID, s.Name
-> HAVING COUNT(DISTINCT e.CourseID) =
-> (
->     SELECT COUNT(*)
->     FROM COURSE
-> );
```

Name
Rahul
Kiran

2 rows in set (0.02 sec)

8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

```
mysql> INSERT INTO STUDENT (SID, Name, DOB, DeptID) VALUES
-> (101, 'Rahul', '2004-01-15', 1),
-> (102, 'Anita', '2003-11-20', 2),
-> (103, 'Kiran', '2004-03-10', 1);
ERROR 1046 (3D000): No database selected
mysql> use co2_at1;
Database changed
mysql> INSERT INTO STUDENT (SID, Name, DOB, DeptID) VALUES
-> (101, 'Rahul', '2004-01-15', 1),
-> (102, 'Anita', '2003-11-20', 2),
-> (103, 'Kiran', '2004-03-10', 1);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO STUDENT (SID, Name, DOB, DeptID)
-> VALUES (104, 'Priya', '2004-06-15', 1);
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE STUDENT
-> SET DeptID = 2
-> WHERE SID = 104;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM STUDENT;
```

SID	Name	DOB	DeptID
101	Rahul	2004-01-15	1
102	Anita	2003-11-20	2
103	Kiran	2004-03-10	1
104	Priya	2004-06-15	2

4 rows in set (0.00 sec)

```
mysql> DELETE FROM STUDENT
-> WHERE SID = 104;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> ^C
```

```
mysql> SELECT * FROM STUDENT;
```

SID	Name	DOB	DeptID
101	Rahul	2004-01-15	1
102	Anita	2003-11-20	2
103	Kiran	2004-03-10	1

```
3 rows in set (0.00 sec)
```

9. Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

```
mysql> SELECT * FROM department;
```

DeptID	DeptName
1	HR
2	IT
3	Finance

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Employee;
```

EmpID	EmpName	Salary	DeptID
101	Rahul	50000.00	1
102	Anita	60000.00	2
103	Kiran	45000.00	1
104	Priya	70000.00	3

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT e.EmpID, e.EmpName, e.Salary, d.DeptName
-> FROM Employee e
-> INNER JOIN Department d
-> ON e.DeptID = d.DeptID
-> WHERE d.DeptName = 'IT'
-> AND e.Salary > 50000;
```

EmpID	EmpName	Salary	DeptName
102	Anita	60000.00	IT

```
1 row in set (0.00 sec)
```

10.Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

```
mysql> CREATE TABLE FLIGHT (  
->     FlightID INT PRIMARY KEY,  
->     FlightName VARCHAR(50),  
->     Source VARCHAR(30),  
->     Destination VARCHAR(30),  
->     AirlineID INT,  
->     FOREIGN KEY (AirlineID) REFERENCES AIRLINE(AirlineID)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> CREATE TABLE PASSENGER (  
->     PassengerID INT PRIMARY KEY,  
->     PassengerName VARCHAR(50),  
->     Age INT  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> CREATE TABLE RESERVATION (  
->     ReservationID INT PRIMARY KEY,  
->     PassengerID INT,  
->     FlightID INT,  
->     TravelDate DATE,  
->     FOREIGN KEY (PassengerID) REFERENCES PASSENGER(PassengerID),  
->     FOREIGN KEY (FlightID) REFERENCES FLIGHT(FlightID)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> INSERT INTO AIRLINE VALUES  
-> (1,'IndiGo'),  
-> (2,'Air India');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO FLIGHT VALUES  
-> (101,'6E201','Chennai','Delhi',1),  
-> (102,'AI305','Mumbai','Bangalore',2);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO AIRLINE VALUES
    -> (1,'IndiGo'),
    -> (2,'Air India');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO FLIGHT VALUES
    -> (101,'6E201','Chennai','Delhi',1),
    -> (102,'AI305','Mumbai','Bangalore',2);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO PASSENGER VALUES
    -> (1,'Rahul',22),
    -> (2,'Anita',21),
    -> (3,'Kiran',23);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO RESERVATION VALUES
    -> (1001,1,101,'2026-08-10'),
    -> (1002,2,102,'2026-08-12'),
    -> (1003,3,101,'2026-08-15');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT p.PassengerName, f.FlightName, f.Source, f.Destination
    -> FROM PASSENGER p
    -> INNER JOIN RESERVATION r
    -> ON p.PassengerID = r.PassengerID
    -> INNER JOIN FLIGHT f
    -> ON r.FlightID = f.FlightID;
```

PassengerName	FlightName	Source	Destination
Rahul	6E201	Chennai	Delhi
Kiran	6E201	Chennai	Delhi
Anita	AI305	Mumbai	Bangalore

3 rows in set (0.00 sec)

```
mysql> SELECT PassengerName
    -> FROM PASSENGER
    -> WHERE PassengerID IN
    -> (
    ->     SELECT PassengerID
    ->     FROM RESERVATION
    ->     WHERE FlightID =
    ->     (
    ->         SELECT FlightID
    ->         FROM FLIGHT
    ->         WHERE FlightName = '6E201'
    ->     )
    -> );
```

PassengerName
Rahul
Kiran

2 rows in set (0.01 sec)

```
mysql> SELECT a.AirlineName, f.FlightName, f.Source, f.Destination
-> FROM AIRLINE a
-> INNER JOIN FLIGHT f
-> ON a.AirlineID = f.AirlineID;
```

AirlineName	FlightName	Source	Destination
IndiGo	6E201	Chennai	Delhi
Air India	AI305	Mumbai	Bangalore

2 rows in set (0.00 sec)

```
mysql> CREATE VIEW FlightDetailsView AS
-> SELECT
->     p.PassengerName,
->     f.FlightName,
->     f.Source,
->     f.Destination,
->     r.TravelDate
-> FROM PASSENGER p
-> JOIN RESERVATION r
-> ON p.PassengerID = r.PassengerID
-> JOIN FLIGHT f
-> ON r.FlightID = f.FlightID;
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> SELECT * FROM FlightDetailsView;
```

PassengerName	FlightName	Source	Destination	TravelDate
Rahul	6E201	Chennai	Delhi	2026-08-10
Kiran	6E201	Chennai	Delhi	2026-08-15
Anita	AI305	Mumbai	Bangalore	2026-08-12

3 rows in set (0.00 sec)